

Fadwa ben Abedall'h

Projet : Application de gestion de bibliothèque musicale avec playlists

Langage utilisé : C (avec gestion dynamique de la mémoire, sans structures)

```
#include <stdio.h>
#include<string.h>
#include <stdlib.h>
#include <time.h>
// Définition des tableaux parallèles
int *ids;
char **titres;
char **artistes;
int *durees;
char **genres;
int *annees;
int *nb_ecoutes;
int nb_morceaux = 0;
int capacite =20;
// Fonction pour initialiser la bibliothèque
void initialiserBibliotheque() {
    ids = (int *)malloc(capacite * sizeof(int));
    titres = (char **)malloc(capacite * sizeof(char *));
    artistes = (char **)malloc(capacite * sizeof(char *));
    durees = (int *)malloc(capacite * sizeof(int));
    genres = (char **)malloc(capacite * sizeof(char *));
    annees = (int *)malloc(capacite * sizeof(int));
    nb_ecoutes = (int *)malloc(capacite * sizeof(int));

    if (!ids || !titres || !artistes || !durees || !genres || !annees || !nb_ecoutes) {
        printf("Erreur d'allocation mémoire\n");
        exit(1);
    }
}
//Fonction permettant d'ajouter un morceau
void ajouterMorceau(int id, const char *titre, const char *artiste, int duree, const char *genre,
int annee) {
    if (nb_morceaux >= capacite) {
        capacite *= 2;
        ids = (int *)realloc(ids, capacite * sizeof(int));
        titres = (char **)realloc(titres, capacite * sizeof(char *));
        artistes = (char **)realloc(artistes, capacite * sizeof(char *));
        durees = (int *)realloc(durees, capacite * sizeof(int));
        genres = (char **)realloc(genres, capacite * sizeof(char *));
```

```

annees = (int *)realloc(annees, capacite * sizeof(int));
nb_ecoutes = (int *)realloc(nb_ecoutes, capacite * sizeof(int));

if (!ids || !titres || !artistes || !durees || !genres || !annees || !nb_ecoutes) {
    printf("Erreur de réallocation mémoire\n");
    exit(1);
}
}

ids[nb_morceaux] = id;
titres[nb_morceaux] = strdup(titre);
artistes[nb_morceaux] = strdup(artiste);
durees[nb_morceaux] = duree;
genres[nb_morceaux] = strdup(genre);
annees[nb_morceaux] = annee;
nb_ecoutes[nb_morceaux] = 0;

nb_morceaux++;
}
//Fonction permettant de supprimer un morceau
void supprimerMorceau(int id) {
    int i, j, trouve=0;
    for (i = 0; i < nb_morceaux; i++) {
        if (ids[i] == id) {
            trouve=1;
            // Libérer la mémoire des anciennes chaînes
            free(titres[i]);
            free(artistes[i]);
            free(genres[i]);

            for (j = i; j < nb_morceaux - 1; j++) {
                ids[j] = ids[j + 1];
                titres[j] = titres[j + 1];
                artistes[j] = artistes[j + 1];
                durees[j] = durees[j + 1];
                genres[j] = genres[j + 1];
                annees[j] = annees[j + 1];
                nb_ecoutes[j] = nb_ecoutes[j + 1];
            }

            nb_morceaux--;
            printf("Morceau supprimé avec succès.\n");
        }
    }
}

```

```

    if(trouve==0)
    {
        printf("Morceau non trouvé.\n");
    }
}
//Fonction permettant de modifier un morceau
void modifierMorceau(int id, int *ids, char **titres, char **artistes, int *durees, char **genres, int
*annees, int *nb_ecoutes, int nb_morceaux) {
    int i, trouve = 0;
    for (i = 0; i < nb_morceaux; i++) {
        if (ids[i] == id) {
            trouve = 1;
            // Libérer la mémoire des anciennes chaînes
            free(titres[i]);
            free(artistes[i]);
            free(genres[i]);

            // Saisie des nouvelles informations
            printf("Entrez le nouveau titre : ");
            titres[i] = (char *)malloc(50 * sizeof(char));
            if (!titres[i]) {
                printf("Erreur d'allocation mémoire\n");
                exit(1);
            }
            scanf("%s", titres[i]);

            printf("Entrez le nouvel artiste : ");
            artistes[i] = (char *)malloc(50 * sizeof(char));
            if (!artistes[i]) {
                printf("Erreur d'allocation mémoire\n");
                exit(1);
            }
            scanf("%s", artistes[i]);

            printf("Entrez la nouvelle durée : ");
            scanf("%d", &durees[i]);

            printf("Entrez le nouveau genre : ");
            genres[i] = (char *)malloc(30 * sizeof(char));
            if (!genres[i]) {
                printf("Erreur d'allocation mémoire\n");
                exit(1);
            }
            scanf("%s", genres[i]);

```

```

        printf("Entrez la nouvelle année de sortie : ");
        scanf("%d", &annees[i]);

        printf("Informations du morceau modifiées avec succès.\n");
        break; // On sort de la boucle une fois le morceau trouvé et modifié
    }
}

if(trouve == 0) {
    printf("Morceau non trouvé.\n");
}
}

//Fonction permettant de rechercher un morceau selon l'artiste ou le titre
void rechercherMorceau() {
    char recherche[50];
    int i, trouve = 0;

    printf("Entrez le titre ou l'artiste à rechercher : ");
    fgets(recherche, sizeof(recherche), stdin);
    recherche[strcspn(recherche, "\n")] = '\0'; // Enlever le saut de ligne à la fin

    for (i = 0; i < nb_morceaux; i++) {
        if (strstr(titres[i], recherche) != NULL || strstr(artistes[i], recherche) != NULL) {
            printf("Morceau trouvé : ID: %d, Titre: %s, Artiste: %s\n", ids[i], titres[i], artistes[i]);
            trouve = 1;
        }
    }

    if (!trouve) {
        printf("Aucun morceau trouvé.\n");
    }
}

//Fonction permettant de filtrer les morceaux selon le genre musical
void filtrerParGenre() {
    char genreRechercher[30];
    int i, trouve = 0;

    printf("Entrez le genre musical à rechercher : ");
    fgets(genreRechercher, sizeof(genreRechercher), stdin);
    genreRechercher[strcspn(genreRechercher, "\n")] = '\0'; // Enlever le saut de ligne à la fin

    for (i = 0; i < nb_morceaux; i++) {
        if (genres[i] != NULL && strstr(genres[i], genreRechercher) != NULL) {

```

```

        printf("Morceau trouvé : ID: %d, Titre: %s, Artiste: %s, Genre: %s\n", ids[i], titres[i],
artistes[i], genres[i]);
        trouve = 1;
    }
}

if (!trouve) {
    printf("Aucun morceau trouvé pour ce genre.\n");
}
}

//Trier par popularité (nombre d'écoutes)
void trierParEcoutes() {
    int i, j, temp;
    char *tempTitre, *tempArtiste, *tempGenre;

    for (i = 0; i < nb_morceaux - 1; i++) {
        for (j = i + 1; j < nb_morceaux; j++) {
            if (nb_ecoutes[i] < nb_ecoutes[j]) {
                // Échange les valeurs par adresse
                temp = nb_ecoutes[i];
                nb_ecoutes[i] = nb_ecoutes[j];
                nb_ecoutes[j] = temp;

                temp = ids[i];
                ids[i] = ids[j];
                ids[j] = temp;

                tempTitre = titres[i];
                titres[i] = titres[j];
                titres[j] = tempTitre;

                tempArtiste = artistes[i];
                artistes[i] = artistes[j];
                artistes[j] = tempArtiste;

                temp = durees[i];
                durees[i] = durees[j];
                durees[j] = temp;

                tempGenre = genres[i];
                genres[i] = genres[j];
                genres[j] = tempGenre;

                temp = annees[i];

```

```

        annees[i] = annees[j];
        annees[j] = temp;
    }
}

// Affichage des morceaux triés
printf("Morceaux triés par nombre d'écoutes (du plus populaire au moins populaire) :\n");
for (i = 0; i < nb_morceaux; i++) {
    printf("ID: %d, Titre: %s, Artiste: %s, Genre: %s, Écoutes: %d\n",
        ids[i], titres[i], artistes[i], genres[i], nb_ecoutes[i]);
}
}

//Trier par année de sortie
void trierParAnnee() {
    int i, j, temp;
    char *tempStr;

    for (i = 0; i < nb_morceaux - 1; i++) {
        for (j = i + 1; j < nb_morceaux; j++) {
            if (annees[i] > annees[j]) {
                // Échange des années
                temp = annees[i];
                annees[i] = annees[j];
                annees[j] = temp;

                // Échange des autres informations
                temp = ids[i];
                ids[i] = ids[j];
                ids[j] = temp;
                temp = durees[i];
                durees[i] = durees[j];
                durees[j] = temp;
                temp = nb_ecoutes[i];
                nb_ecoutes[i] = nb_ecoutes[j];
                nb_ecoutes[j] = temp;

                tempStr = titres[i];
                titres[i] = titres[j];
                titres[j] = tempStr;
                tempStr = artistes[i];
                artistes[i] = artistes[j];
                artistes[j] = tempStr;
                tempStr = genres[i];

```

```

        genres[i] = genres[j];
        genres[j] = tempStr;
    }
}

// Affichage après tri
printf("\nMorceaux triés par année de sortie :\n");
for (i = 0; i < nb_morceaux; i++) {
    printf("ID: %d, Titre: %s, Artiste: %s, Année: %d, Genre: %s, Durée: %d sec, Écoutes:
%d\n",
        ids[i], titres[i], artistes[i], annees[i], genres[i], durees[i], nb_ecoutes[i]);
}

const int MAX_PLAYLISTS = 10; // Nombre maximum de playlists
// Tableaux parallèles pour stocker plusieurs playlists
char *Nom_playlists[MAX_PLAYLISTS]; // Tableaux contenant les noms de playlists
int *playlists[MAX_PLAYLISTS]; // Chaque élément est un tableau dynamique de morceaux
int tailles[MAX_PLAYLISTS] = {0}; // Taille actuelle de chaque playlist
int capacites[MAX_PLAYLISTS] = {0}; // Capacité de chaque playlist

// Fonction pour créer une nouvelle playlist
void creerPlaylist(int index, const char *nom) {
    if (index < 0 || index >= MAX_PLAYLISTS) {
        printf("Index de playlist invalide !\n");
    }
    // Vérification si la playlist existe déjà
    if (Nom_playlists[index] != NULL) {
        printf("Erreur : Une playlist existe déjà à l'index %d.\n", index);
    }

    // Vérification du nom
    if (nom == NULL || strlen(nom) == 0) {
        printf("Erreur : Le nom de la playlist ne peut pas être vide.\n");
    }

    capacites[index] = 5; // Capacité initiale
    tailles[index] = 0;
    playlists[index] = (int *)malloc(capacites[index] * sizeof(int));

    if (playlists[index] == NULL) {
        printf("Erreur d'allocation mémoire.\n");
        exit(1);
    }
}

```

```

Nom_playlists[index] = (char *)malloc((strlen(nom) + 1) * sizeof(char));
if (Nom_playlists[index] == NULL) {
    printf("Erreur d'allocation mémoire pour le nom.\n");
    exit(1);
}
strcpy(Nom_playlists[index], nom);

printf("Playlist \"%s\" à l'index %d créée avec succès !\n", nom, index);
}
// Fonction pour ajouter un morceau à une playlist
void ajouterMorceauPlaylist(int index, int morceauID) {
    // Vérification de l'index de la playlist
    if (index < 0 || index >= MAX_PLAYLISTS || playlists[index] == NULL) {
        printf("Playlist invalide !\n");
    }
    else {
        // Vérifier si la capacité est suffisante
        if (tailles[index] >= capacites[index]) {
            capacites[index] *= 2; // Double la capacité
            playlists[index] = (int *)realloc(playlists[index], capacites[index] * sizeof(int));

            if (playlists[index] == NULL) {
                printf("Erreur de réallocation mémoire.\n");
                exit(1);
            }
        }

        // Ajouter le morceau
        playlists[index][tailles[index]] = morceauID;
        tailles[index]++;
        printf("Morceau %d ajouté à la playlist %d.\n", morceauID, index);
    }
}
//Fonction permettant de supprimer un morceau d'une playlist
void supprimerMorceauPlaylist(int index, int morceauID) {
    // Vérification de l'index de la playlist
    if (index < 0 || index >= MAX_PLAYLISTS) {
        printf("Index de playlist invalide.\n");
    }

    int i, trouve = 0;
    for (i = 0; i < tailles[index]; i++) {
        if (playlists[index][i] == morceauID) {

```

```

        trouve = 1;
        break;
    }
}

if (trouve) {
    for (int j = i; j < tailles[index] - 1; j++) {
        playlists[index][j] = playlists[index][j + 1];
    }
    tailles[index]--;
    printf("Morceau %d supprimé de la playlist %d.\n", morceauID, index);
} else {
    printf("Morceau non trouvé.\n");
}
}

// Fonction pour afficher toutes les playlists
void afficherPlaylists() {
    for (int i = 0; i < MAX_PLAYLISTS; i++) {
        if (playlists[i] != NULL) {
            printf("Playlist %d: ", i);
            for (int j = 0; j < tailles[i]; j++) {
                printf("%d ", playlists[i][j]);
            }
            printf("\n");
        }
    }
}

// Réorganise les morceaux dans une même playlist en échangeant deux positions
void reorganiserPlaylist(int index, int pos1, int pos2) {
    if (index < 0 || index >= MAX_PLAYLISTS) {
        printf("Index de playlist invalide.\n");
    }
    else if (pos1 < 0 || pos2 < 0 || pos1 >= tailles[index] || pos2 >= tailles[index]) {
        printf("Position invalide dans la playlist.\n");
    }
    else {
        // Échange les deux morceaux
        int temp = playlists[index][pos1];
        playlists[index][pos1] = playlists[index][pos2];
        playlists[index][pos2] = temp;

        printf("Les morceaux ont été échangés avec succès.\n");
    }
}

```

```

}
//Calculer la durée totale d'une playlist
int calculerDureePlaylist(int *playlist[], int index, int taille, int *durees, int *ids, int nb_Morceaux) {
    int dureeTotale = 0;

    for (int i = 0; i < taille; i++) {
        int id = playlist[index][i]; // Récupère l'ID du morceau dans la playlist

        for (int j = 0; j < nb_Morceaux; j++) {
            if (ids[j] == id) { // Vérifie si l'ID correspond à celui dans la liste de morceaux
                dureeTotale += durees[j]; // Ajoute la durée du morceau
                break; // Sort de la boucle dès qu'on trouve le morceau correspondant
            }
        }
    }

    return dureeTotale;
}

//Générer des playlists automatiques
void genererPlaylistAuto(int indexPlaylist, const char* genreCible) {
    // Vérifier que l'index est valide
    if (indexPlaylist < 0 || indexPlaylist >= MAX_PLAYLISTS) {
        printf("Index de playlist invalide!\n");
        return;
    }

    // Réinitialiser la playlist
    tailles[indexPlaylist] = 0;

    // Parcourir tous les morceaux
    for (int i = 0; i < nb_morceaux; i++) {
        // Comparaison des genres (strcmp retourne 0 si égaux)
        if (strcmp(genres[i], genreCible) == 0) {
            // Vérifier si on a besoin de plus d'espace
            if (tailles[indexPlaylist] >= capacites[indexPlaylist]) {
                capacites[indexPlaylist] *= 2;
                playlists[indexPlaylist] = realloc(playlists[indexPlaylist], capacites[indexPlaylist] *
sizeof(int));
                if (!playlists[indexPlaylist]) {
                    printf("Erreur de réallocation mémoire!\n");
                    exit(1);
                }
            }
            // Ajouter l'ID du morceau à la playlist

```

```

        playlists[indexPlaylist][tailles[indexPlaylist]] = ids[i];
        tailles[indexPlaylist]++;
    }
}

printf("Playlist générée avec %d morceaux du genre %s\n", tailles[indexPlaylist],
genreCible);
}
//Simuler la lecture séquentielle ou aléatoire d'une playlist
void simulerLecture(int modeAleatoire, int index) {
    if (modeAleatoire) {
        srand(time(NULL));
        printf("Lecture aléatoire :\n");
        for (int i = 0; i < tailles[index]; i++) {
            int randomIndex = rand() % tailles[index];
            printf("Lecture du morceau ID: %d\n", playlists[index][randomIndex]);
        }
    } else {
        printf("Lecture normale :\n");
        for (int i = 0; i < tailles[index]; i++) {
            printf("Lecture du morceau ID: %d\n", playlists[index][i]);
        }
    }
}
//Recommander des morceaux similaires basés sur le genre
int recommanderMorceau(int *playlist, char **genres, int nbMorceaux, const char *genreCible) {
    for (int i = 0; i < nbMorceaux; i++) {
        int deja_Present = 0;

        if (strcmp(genres[i], genreCible) == 0) {
            // Calculer la taille de la playlist (supposée terminée par -1)
            for (int j = 0; playlist[j] != -1; j++) {
                if (playlist[j] == i) {
                    deja_Present = 1;
                    break;
                }
            }
        }

        if (!deja_Present) {
            return i;
        }
    }
}
return -1;

```

```

}
//Libération de la mémoire allouée dynamiquement
void libererMemoire() {
    // Libérer les morceaux
    for (int i = 0; i < nb_morceaux; i++) {
        if (titres[i] != NULL) {
            free(titres[i]);
        }
        if (artistes[i] != NULL) {
            free(artistes[i]);
        }
        if (genres[i] != NULL) {
            free(genres[i]);
        }
    }
    if (ids != NULL) {
        free(ids);
    }
    if (titres != NULL) {
        free(titres);
    }
    if (artistes != NULL) {
        free(artistes);
    }
    if (durees != NULL) {
        free(durees);
    }
    if (genres != NULL) {
        free(genres);
    }
    if (annees != NULL) {
        free(annees);
    }
    if (nb_ecoutes != NULL) {
        free(nb_ecoutes);
    }

    // Libérer les playlists
    for (int i = 0; i < MAX_PLAYLISTS; i++) {
        if (playlists[i] != NULL) {
            free(playlists[i]);
        }
        if (Nom_playlists[i] != NULL) {
            free(Nom_playlists[i]);
        }
    }
}

```

```

    }
}

printf("Fermeture du programme...\n");
}
// Fonction principale
int main() {
    int choix, index, id, Durée, année, reponse1, genreCible, modeAleatoire, pos1, pos2 ;
    char reponse ;
    char *titre;
    char*artiste ;
    char*genre;
    char nom[30];

    initialiserBibliotheque() ;
    while (1) {
        printf("\nGestionnaire de Playlist Musicale\n");
        printf("1. ajouter un morceau \n");
        printf("2. Afficher le morceau \n");
        printf("3. Recherche de morceaux:\n");
        printf("4. supprimer le morceau \n");
        printf("5. créer une playlist \n");
        printf("6. Ajouter un morceau à un playlist\n" ) ;
        printf("7. Afficher playlists\n" ) ;
        printf("8. Recommander un morceau\n");
        printf("9. Supprimer de morceau dans une playlist\n");
        printf("10.Quitter\n") ;
        printf("Choix : ");
        scanf("%d", &choix);

        switch (choix) {
            case 1:
                titre = (char *)malloc(100 * sizeof(char));
                artiste = (char *)malloc(100 * sizeof(char));
                genre = (char *)malloc(100 * sizeof(char));
                if (!titre || !artiste || !genre) {
                    printf("Erreur d'allocation mémoire pour les chaînes\n");
                    return 1;
                }
                printf("ID du morceau : ");
                scanf("%d", &id);
                while (getchar() != '\n'); // vider le buffer

                printf("Titre : ");

```

```

fgets(titre, 100, stdin);
titre[strcspn(titre, "\n")] = '\0'; // enlever le \n

printf("Artiste : ");
fgets(artiste, 100, stdin);
artiste[strcspn(artiste, "\n")] = '\0';

printf("Durée (secondes) : ");
scanf("%d", &Durée);
while (getchar() != '\n');

printf("Genre : ");
fgets(genre, 100, stdin);
genre[strcspn(genre, "\n")] = '\0';

printf("Année de sortie : ");
scanf("%d", &année );

ajouterMorceau(id, titre, artiste, Durée, genre, année);

// Libération mémoire
free(titre);
free(artiste);
free(genre);
printf ("\nMorceau ajouté avec succès !\n") ;
break;
case 2:
printf ("Afficher les morceaux selon:\n" ) ;
printf (" 1.selon le nombre d'écoute\n") ;
printf (" 2.selon l'année de sortie\n") ;
printf("Choisissez une option : ");
scanf("%d", &reponse1) ;
if(reponse1==1)
{
    trierParEcoutés() ;
}
else if (reponse1 == 2)
{
    trierParAnnee() ;
}
else {
    printf("Choix invalide.\n");
}
break;
case 3: {

```

```

printf("Recherche de morceaux:\n");
printf(" 1. Par genre\n");
printf(" 2. Par titre ou artiste\n");
printf("Choisissez une option : ");
scanf("%d", &reponse1); // Ajout du & manquant
while(getchar() != '\n'); // Vider le buffer

if (reponse1 == 1) {
    filtrerParGenre();
} else if (reponse1 == 2) {
    rechercherMorceau();
} else {
    printf("Choix invalide.\n");
}
break;
}

case 4:
    printf("ID du morceau : ");
    scanf("%d", &id);
    supprimerMorceau(id);
    break;
case 5:
    while (getchar() != '\n'); // vide le tampon avant de lire avec fgets
    printf("Donner le nom de la playlist : ");
    fgets(nom, sizeof(nom), stdin);

// Retirer le saut de ligne si présent

printf("Index de la nouvelle playlist (0-%d) : ", MAX_PLAYLISTS - 1);
scanf("%d", &index);

creerPlaylist(index, nom);

printf("Générer automatiquement une playlist ? (y/n) : ");
scanf(" %c", &reponse);

if (reponse == 'y' || reponse == 'Y') {
    char genreCible[30];
    printf("Entrez le genre cible : ");
    scanf("%s", genreCible);
    genererPlaylistAuto(index, genreCible);
    printf("Playlist générée automatiquement !\n");
}

```

```
break;
case 6:
```

```
    printf("Index de la playlist (0-%d) : ", MAX_PLAYLISTS - 1);
    scanf("%d", &index);
    printf("ID du morceau à ajouter : ");
    scanf("%d", &id);
    ajouterMorceauPlaylist(index, id);
    printf("Réorganiser la playlist ? (y/n) :\n ");
    scanf(" %c", &reponse);
    if (reponse == 'y' || reponse == 'Y') {
        printf ("Donner la première position du premier morceau. ") ;
        scanf ("%d", &pos1) ;
        printf ("Donner la deuxième position du deuxième morceau.") ;
        scanf ("%d", &pos2) ;
        reorganiserPlaylist(index, pos1, pos2);
    }
    break;
```

```
case 7:
```

```
    afficherPlaylists();
    printf("Simuler la lecture d'une playlist ? (y/n) : ");
    scanf(" %c", &reponse);
    if (reponse == 'y' || reponse == 'Y') {

        printf("Index de la playlist (0-%d) : ", MAX_PLAYLISTS - 1);
        scanf("%d", &index);
        printf("1. Lecture aléatoire ");
        printf("0.Lecture normale ");
        scanf("%d", &modeAleatoire);
        simulerLecture(modeAleatoire, index);
    }
    break;
```

```
case 8: {
```

```
    char genreCible[30];
    printf("Genre cible : ");
    scanf("%s", genreCible);
```

```
    printf("Index de la playlist (0-%d) : ", MAX_PLAYLISTS - 1);
    scanf("%d", &index);
```

```
    if (index >= 0 && index < MAX_PLAYLISTS) {
        int indice = recommanderMorceau(playlists[index], genres, nb_morceaux,
genreCible);
        if (indice != -1) {
```

```

        printf("Morceau recommandé: ID: %d, Titre: %s\n", ids[indice], titres[indice]);
    } else {
        printf("Aucun morceau trouvé.\n");
    }
}
break;
}
case 9 : {
    printf("ID du morceau : ");
    scanf("%d", &id);
    supprimerMorceauPlaylist(index, id) ;
    break ;
}
case 10:
    libererMemoire() ;
    return 0;
default:
    printf("Choix invalide !\n");
}
}

return 0;
}

```